

Hanoi University of Science and Technology
School of Information and Communications Technology



COURSE REPORT
AIMS System Development

Student: Tran Viet Cuong

Student ID: 202416784

Supervisor: Ho Viet Duc Luong

Department: Object-Oriented Programming Practice

HANOI, DECEMBER 2025

Contents

1. Use Case Diagram of the AIMS System.....	4
1.1 General Introduction and Objectives	4
1.2 Actors	4
1.3 Use Cases	4
1.4 Use Case Diagram.....	5
1.5 Analysis and Evaluation.....	6
1.5.1 Remarks	6
1.5.2 Conclusion	6
2. Class Diagram of the AIMS System	7
2.1 Purpose.....	7
2.2 Diagram.....	7
2.3 Analysis and Remarks	8
2.4 Brief Description of Classes	8
2.5 Conclusion	9
3. Use Application.....	10
3.1 Application Startup	10
3.2 Add to Cart Functionality.....	11
3.3 Play Button Functionality	13
3.4 View Cart Button Functionality	15
3.4.1 Filtering and Searching.....	16
3.4.2 Order Placement.....	18
3.4.3 Returning to Store View.....	20
3.4.4 Item Selection and Removal	21
3.5 Add to Store Functionality (Manager)	23
4. Exception Handling in the AIMS System.....	27
4.1 Purpose.....	27

4.2 Common Exception Scenarios.....	27
4.3 Implementing Exception Handling in Code	28
4.3.1 Handling Media Playback Errors.....	28
4.3.2 Handling Cart Limit and Duplicate Items	28
4.4 Benefits of Exception Handling.....	29
4.5 Conclusion	29

1. Use Case Diagram of the AIMS System

1.1 General Introduction and Objectives

The Use Case Diagram of the AIMS (An Internet Media Store) system illustrates the primary functionalities from the users' perspective. The system is designed to support the complete lifecycle of media consumption, including browsing, sophisticated cart management, and administrative inventory control.

The main objective of this diagram is to define the functional scope of the AIMS application and clearly visualize the interactions between human actors and external systems.

1.2 Actors

- **Customer:** The primary end-user who browses the catalog, manages a personal shopping cart, and initiates the purchasing process.
- **Store Manager:** An administrative actor responsible for maintaining the store's inventory and overseeing pending orders.
- **Card Association System:** An external system that handles the verification and processing of financial transactions during the payment stage.

1.3 Use Cases

The updated functional requirements, reflecting the current system capabilities, are summarized in the table below:

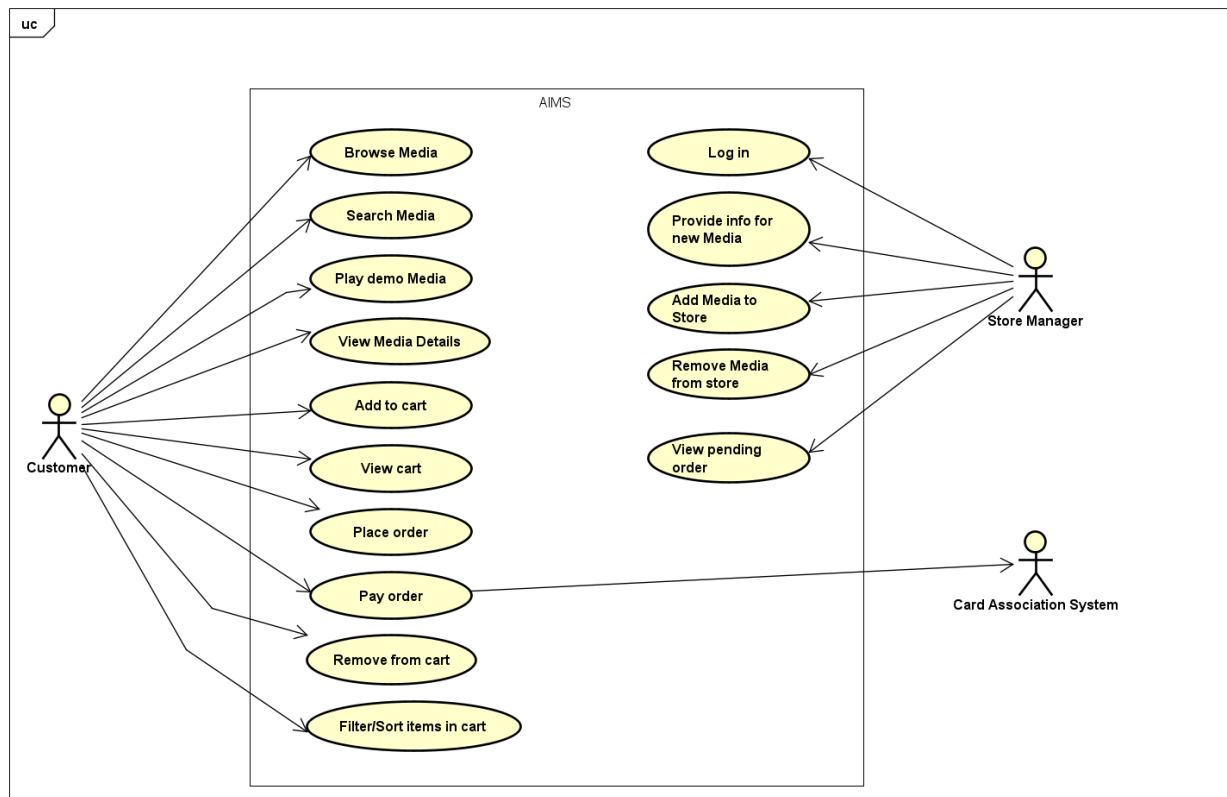
Use Case	Actor	Description
Browse Media	Customer	Browse all available media items (Books, CDs, DVDs) in the store.
Search Media	Customer	Search for specific items within the store catalog.
Play Demo Media	Customer	Preview a demonstration of playable media items.
View Media Details	Customer	View technical details, cost, and category of an item.
Add to Cart	Customer	Add a selected media item to the shopping cart.
View Cart	Customer	Access the shopping cart to review selected items.
Remove from Cart	Customer	Delete specific items from the current cart.

Filter/Sort Cart	Customer	Organize cart items by criteria such as ID, Title, or Cost.
Place Order	Customer	Finalize the selection and submit an order.
Pay Order	Customer / Card System	Complete payment via the external association system.
Log in	Store Manager	Access administrative features through secure authentication.
Add Media to Store	Store Manager	Input data and add new media items to the inventory.
Remove Media	Store Manager	Delete outdated or out-of-stock media from the system.
View Pending Order	Store Manager	Monitor orders awaiting fulfillment or approval.

Table 1: Use Cases of the AIMS System

1.4 Use Case Diagram

The following diagram provides a visual representation of the interactions described above, showcasing the boundaries between the AIMS system and its actors.



1.5 Analysis and Evaluation

1.5.1 Remarks

The diagram effectively separates responsibilities, ensuring a clear division between customer shopping experiences and manager administrative tasks. By generalizing from "DVDs" to "Media," the system demonstrates the flexibility to handle various product types (Books, CDs, DVDs) as defined in the class hierarchy. Furthermore, the inclusion of "Filter/Sort" and "Remove from cart" reflects the realistic requirements of a modern e-commerce application.

1.5.2 Conclusion

The Use Case Diagram serves as the functional blueprint for the AIMS system. It ensures that all core requirements—from browsing to payment—are accounted for, providing a solid foundation for the subsequent Class Diagram design and software implementation.

2. Class Diagram of the AIMS System

2.1 Purpose

The Class Diagram illustrates the static structure of the AIMS system by defining the object classes, their internal attributes, methods, and the complex relationships between them. It serves as the architectural blueprint for the software, ensuring that the implementation follows strict object-oriented principles such as encapsulation, inheritance, and polymorphism to enhance maintainability and extensibility.

2.2 Diagram

The overall system architecture is organized into several key packages: media, cart, store, screen, main, and exception

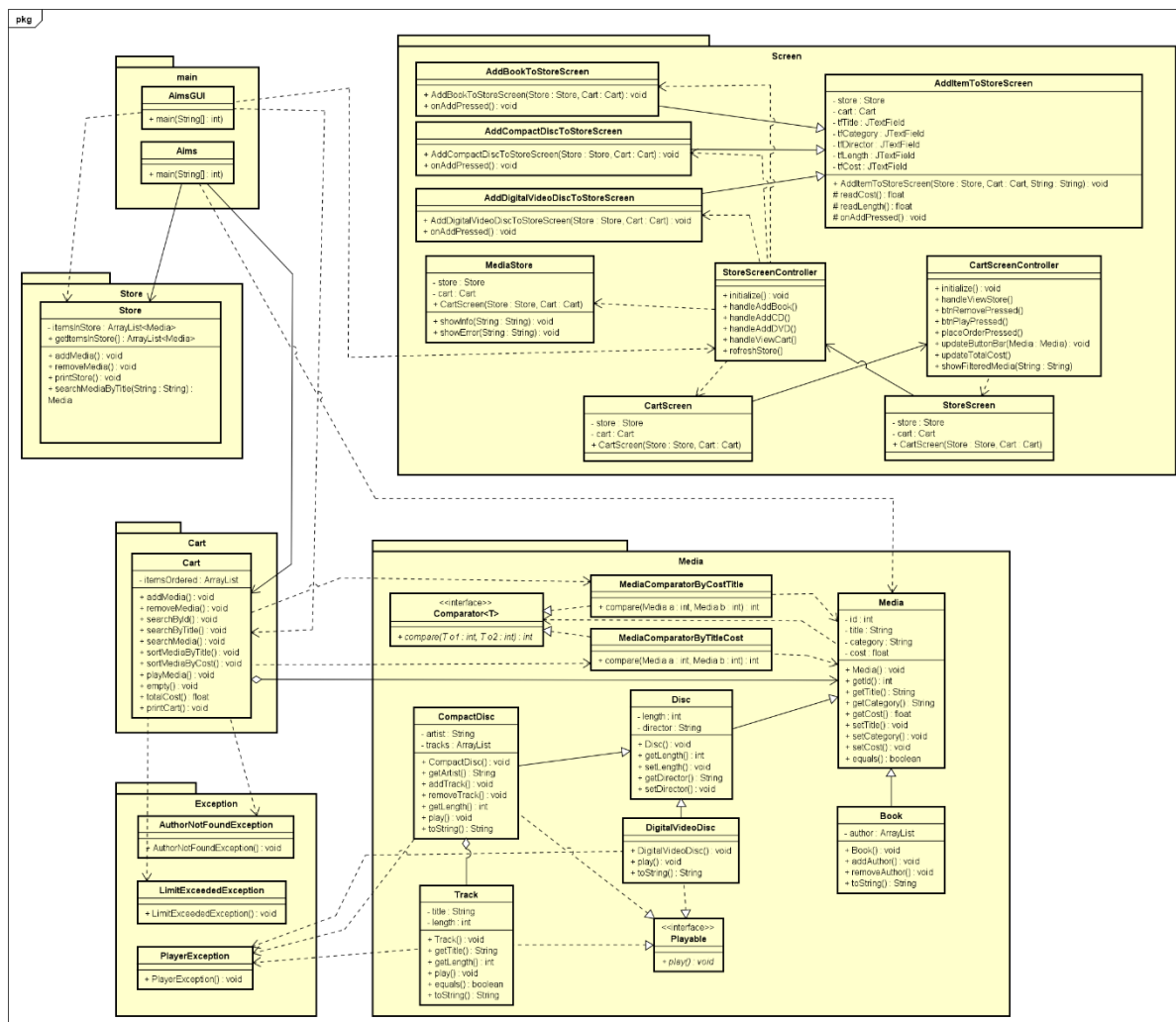


Figure 2: Class Diagram of the AIMS System

2.3 Analysis and Remarks

The system architecture demonstrates a high level of modularity through the following design choices:

- **The Media Hierarchy:** The Media class is defined as an abstract base class. Specific types like Book and Disc inherit from it, while Disc acts as a further specialized parent for DigitalVideoDisc and CompactDisc.
- **Interface Realization:** The Playable interface is implemented by classes that require playback functionality, specifically DigitalVideoDisc, CompactDisc, and Track.
- **Encapsulation of Logic:** The Store class manages the available inventory through an ArrayList<Media>. Meanwhile, the Cart class utilizes an ObservableList<Media> to manage items chosen by the customer, allowing for real-time UI updates.
- **GUI Architecture:** The screen package separates the visual components (e.g., StoreScreen, CartScreen) from their logic handlers (StoreScreenController, CartScreenController).
- **Relationships:**
 - **Inheritance** (Solid line, hollow triangle) is used for the Media hierarchy.
 - **Aggregation** (Solid line, hollow diamond) represents a CompactDisc containing a collection of Track objects.
 - **Dependency** (Dashed line, hollow arrow) indicates that controllers or UI screens utilize classes from other packages for instantiation or data processing

2.4 Brief Description of Classes

The table below summarizes the primary functions of the core classes within the system:

Class	Functionality
Media	Abstract parent class containing shared data like id, title, category, and cost.
Disc	Intermediate class for disc-based media, adding director and length.
DigitalVideoDisc	Represents a DVD and implements playback logic.
CompactDisc	Represents a CD; manages a collection of music tracks.
Track	Represents an individual music track within a CD.
Book	Represents a printed book, specifically managing a list of authors.
Playable	An interface defining the play() method for compatible media types.
Cart	Manages customer selections, including adding, removing, sorting, and filtering.
Store	Manages the store's inventory and supports search and management tasks.

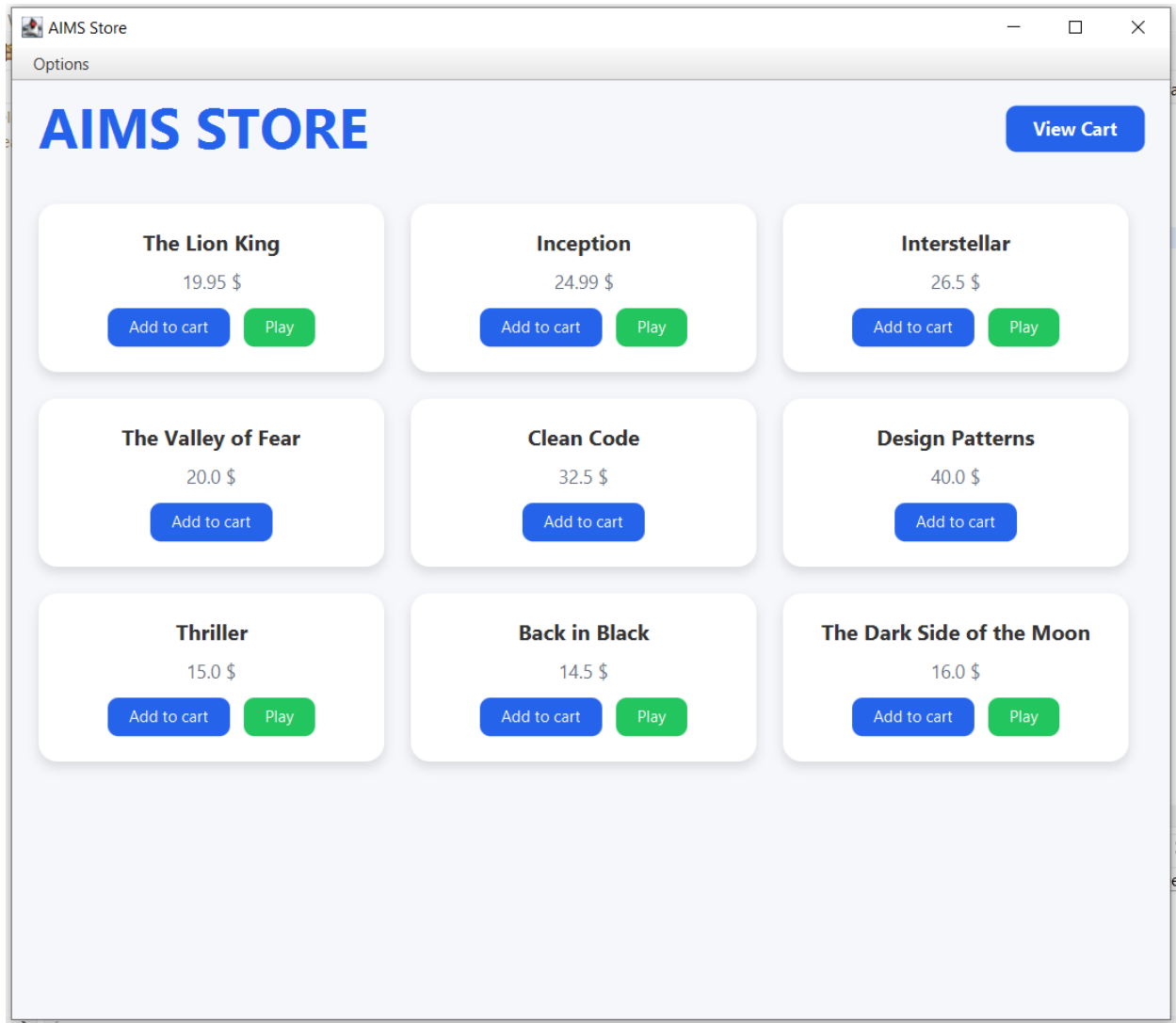
2.5 Conclusion

The Class Diagram accurately reflects the logical requirements of the AIMS system. By utilizing an abstract base class and interfaces, the design allows the system to handle diverse media types through a unified interface, significantly improving the robustness of the code during the implementation phase.

3. Use Application

3.1 Application Startup

Upon launching the application via AimsGUI, the system initializes the Store with a predefined set of media items . The main StoreScreen is then displayed, providing a grid-style interface for browsing .

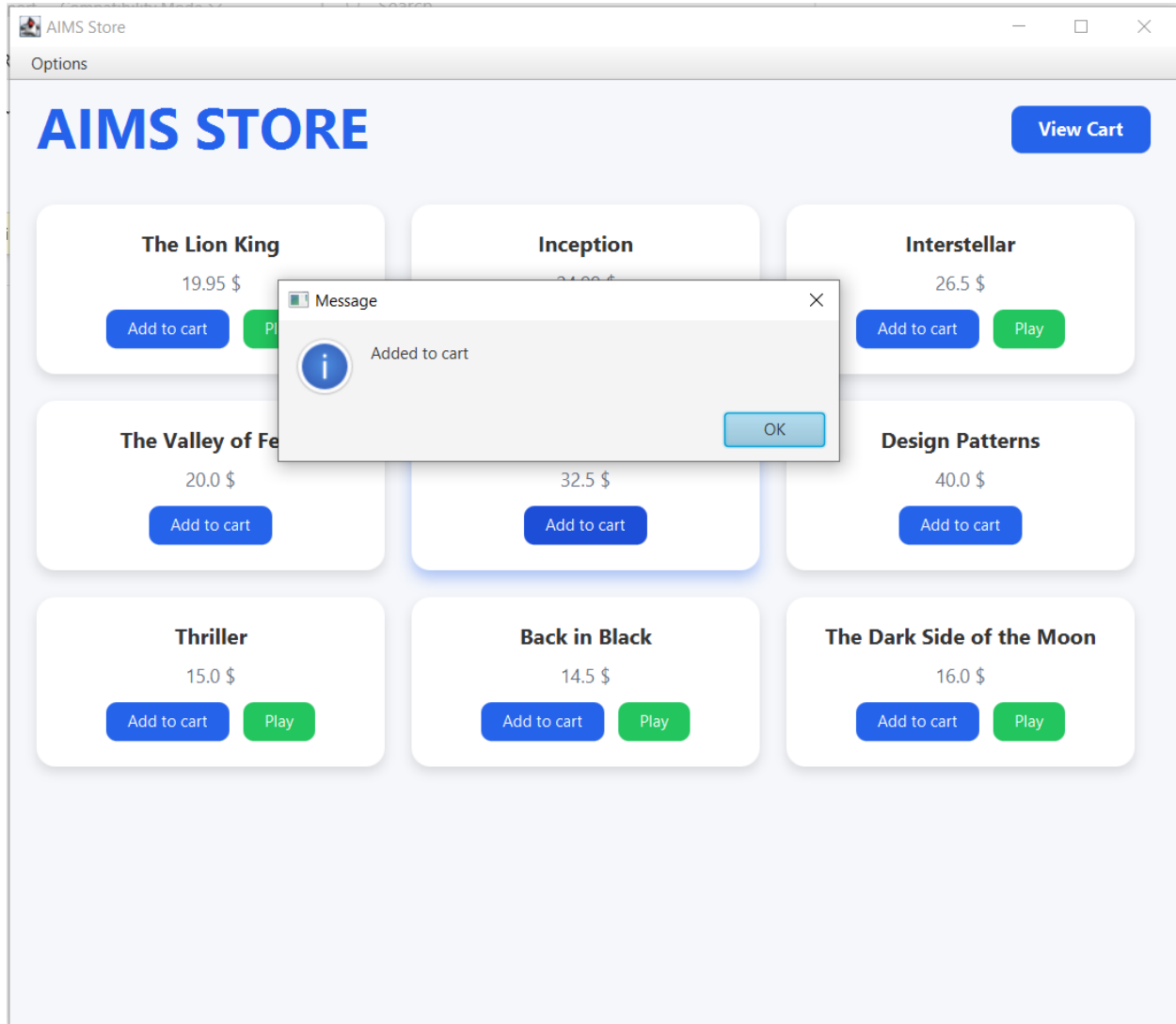


- The interface displays the title and cost for each media item .
- Each card contains an **Add to cart** button.
- The **Play** button is dynamically displayed only if the media implements the Playable interface (e.g., DVDs and CDs) .

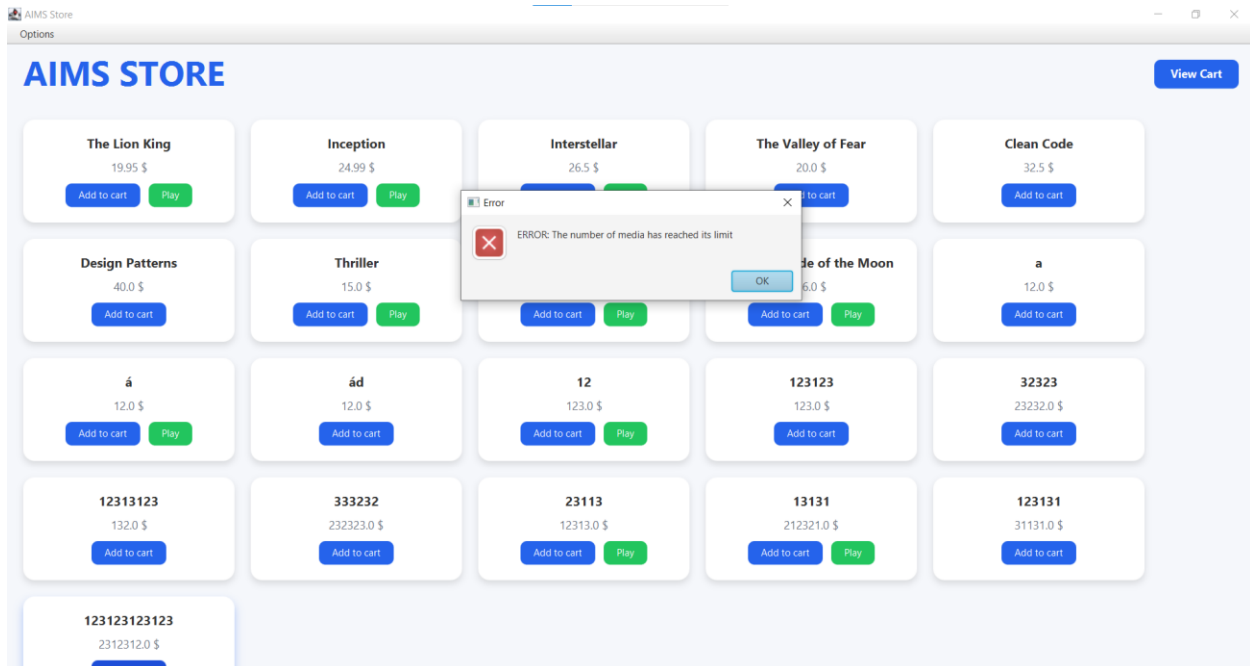
- The **View Cart** button is positioned at the top right of the header for quick access to the shopping list .

3.2 Add to Cart Functionality

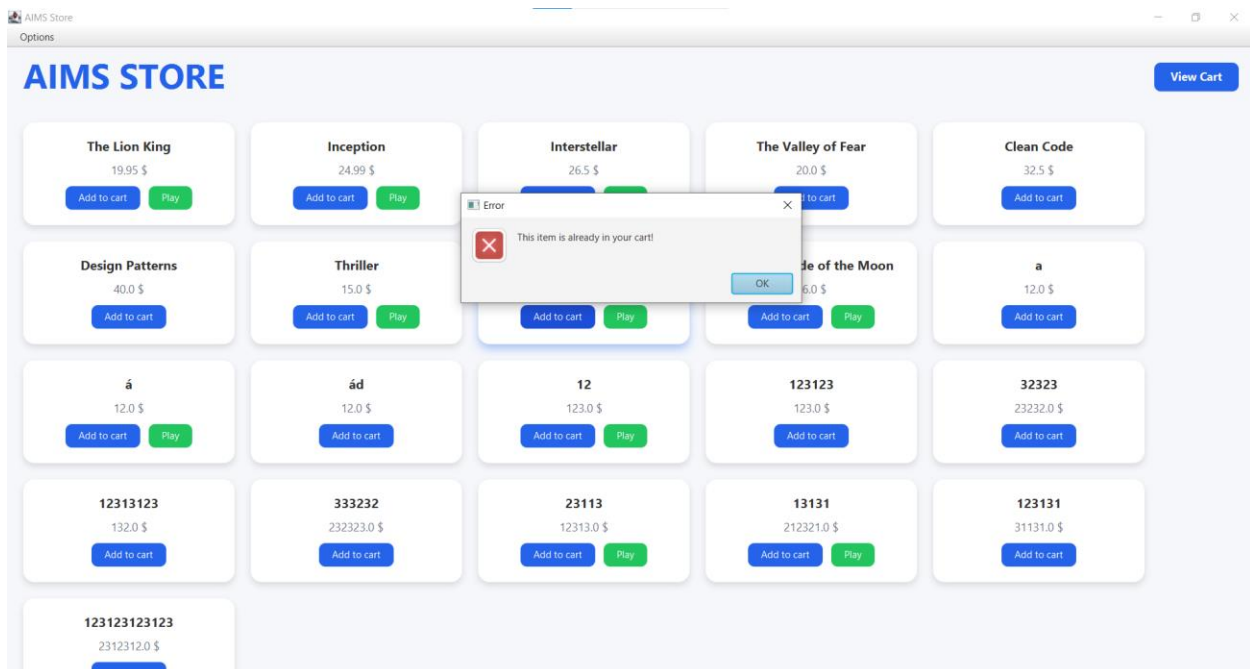
When a user clicks the **Add to cart** button, the `MediaStore` class triggers the `cart.addMedia(media)` method.



- **Success:** A JavaFX Alert is displayed with an information message confirming the item has been added.



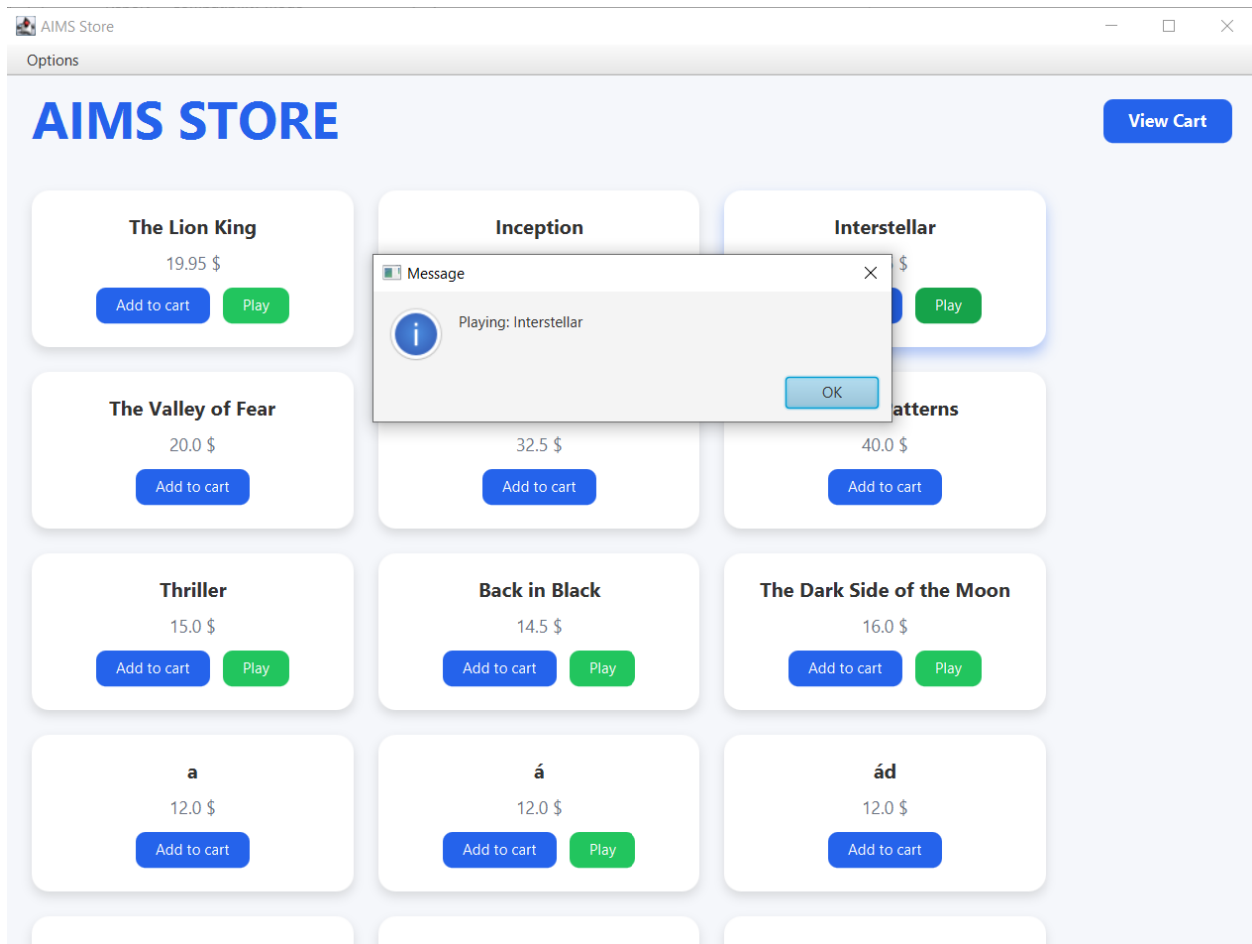
- **Failure (Limit Exceeded):** If the cart has reached its capacity of 20 items, the system catches a `LimitExceededException` and displays an error message.



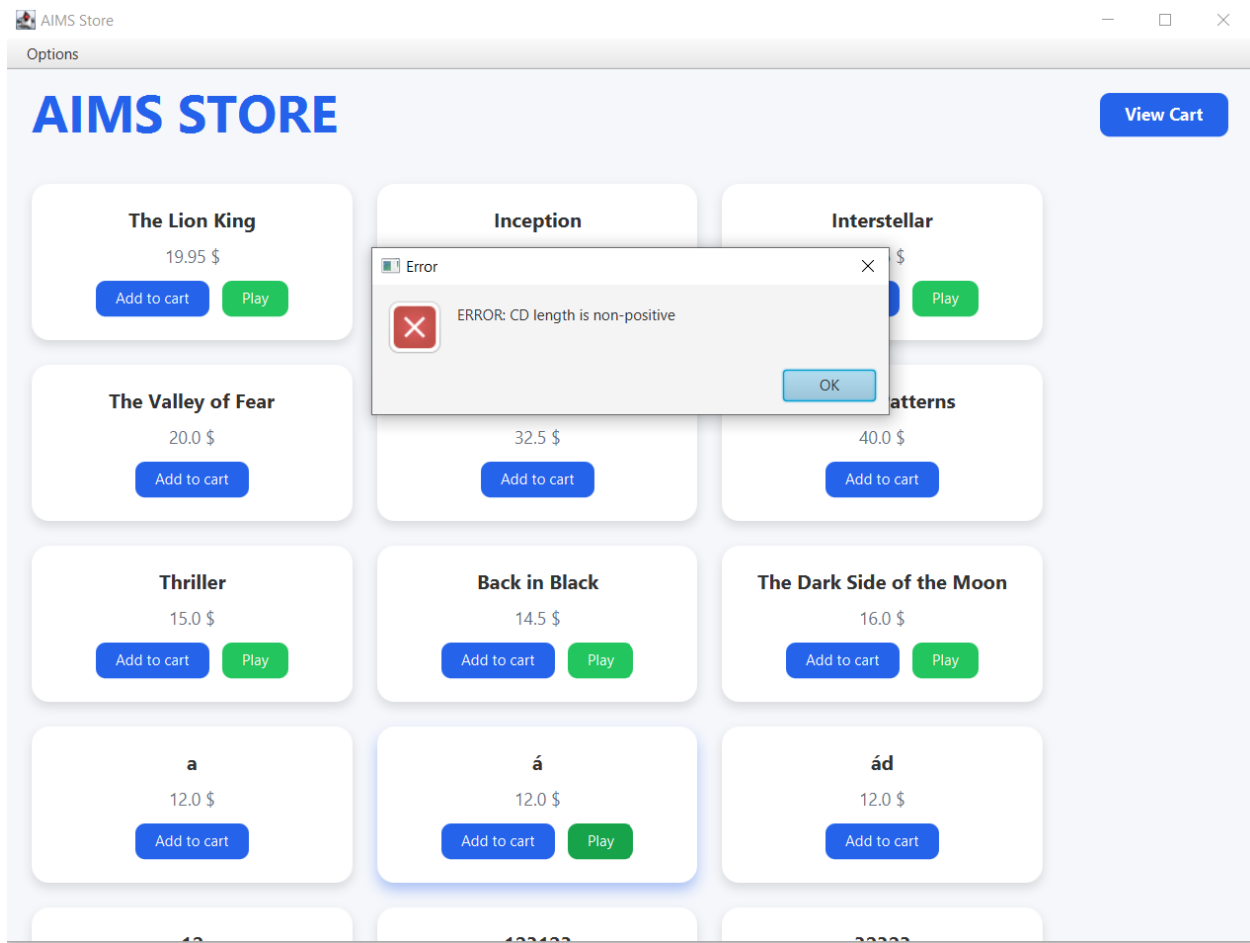
- **Failure (Duplicate):** Although not explicitly a cart exception in this version, the system logic ensures items are managed as an ObservableList

3.3 Play Button Functionality

For playable media like DigitalVideoDisc or CompactDisc, clicking the **Play** button executes the play() method defined in the Playable interface .



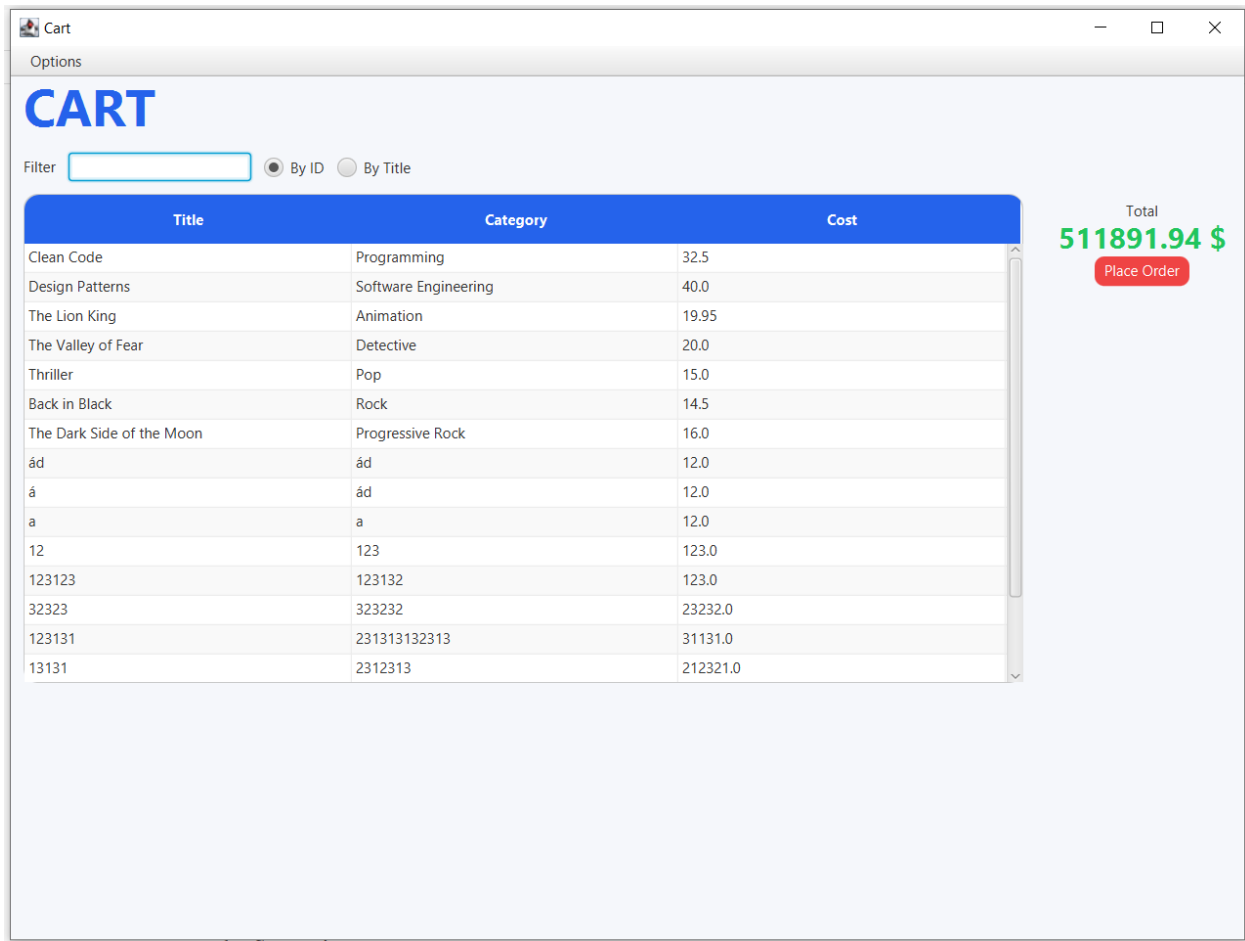
- A dialog box appears showing the playback status (e.g., "Playing: [Title]").



- If the media length is non-positive, a `PlayerException` is caught, and an error alert is shown to the user .

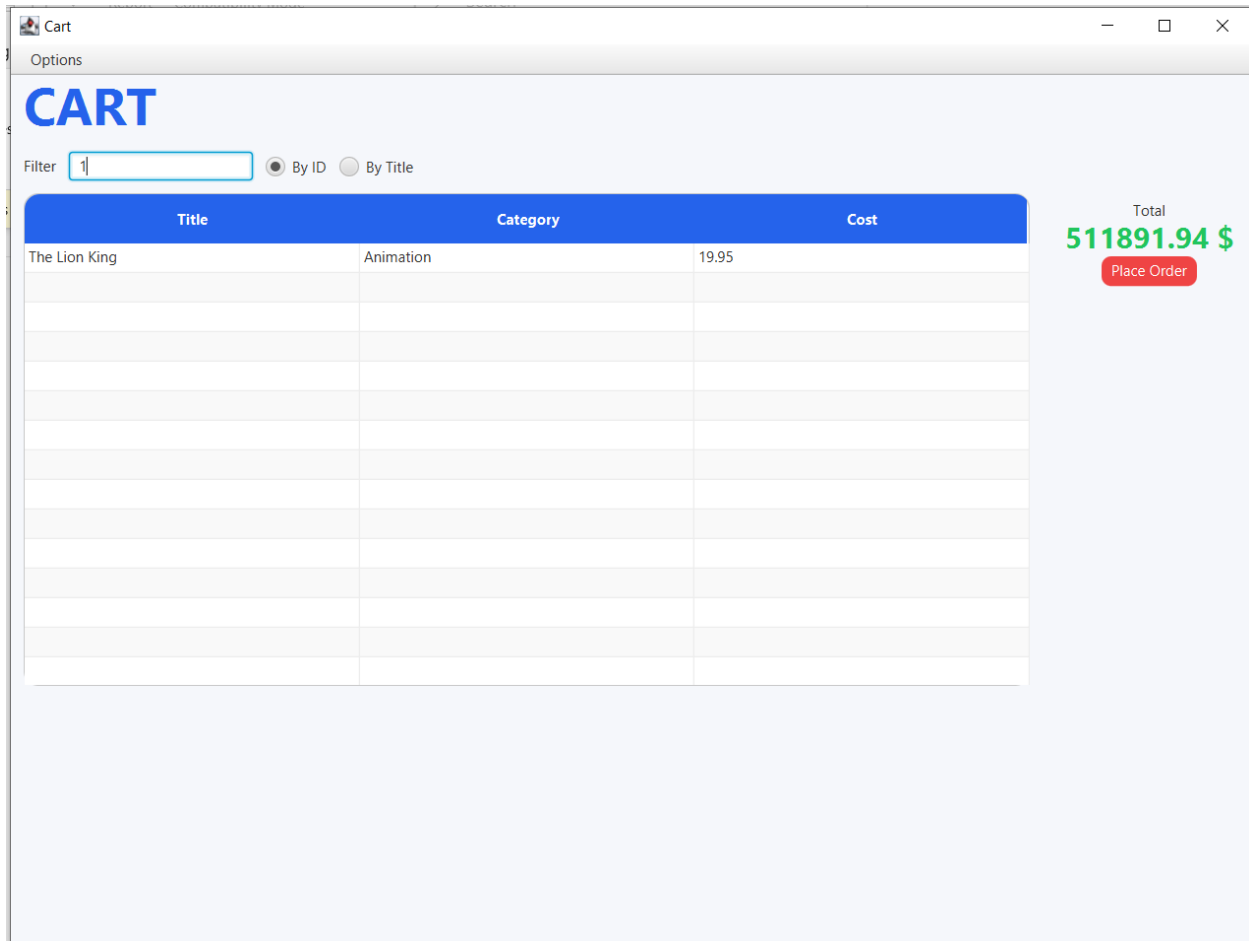
3.4 View Cart Button Functionality

Clicking the **View Cart** button switches the interface to the `CartScreen`. This screen allows users to manage their selected items.

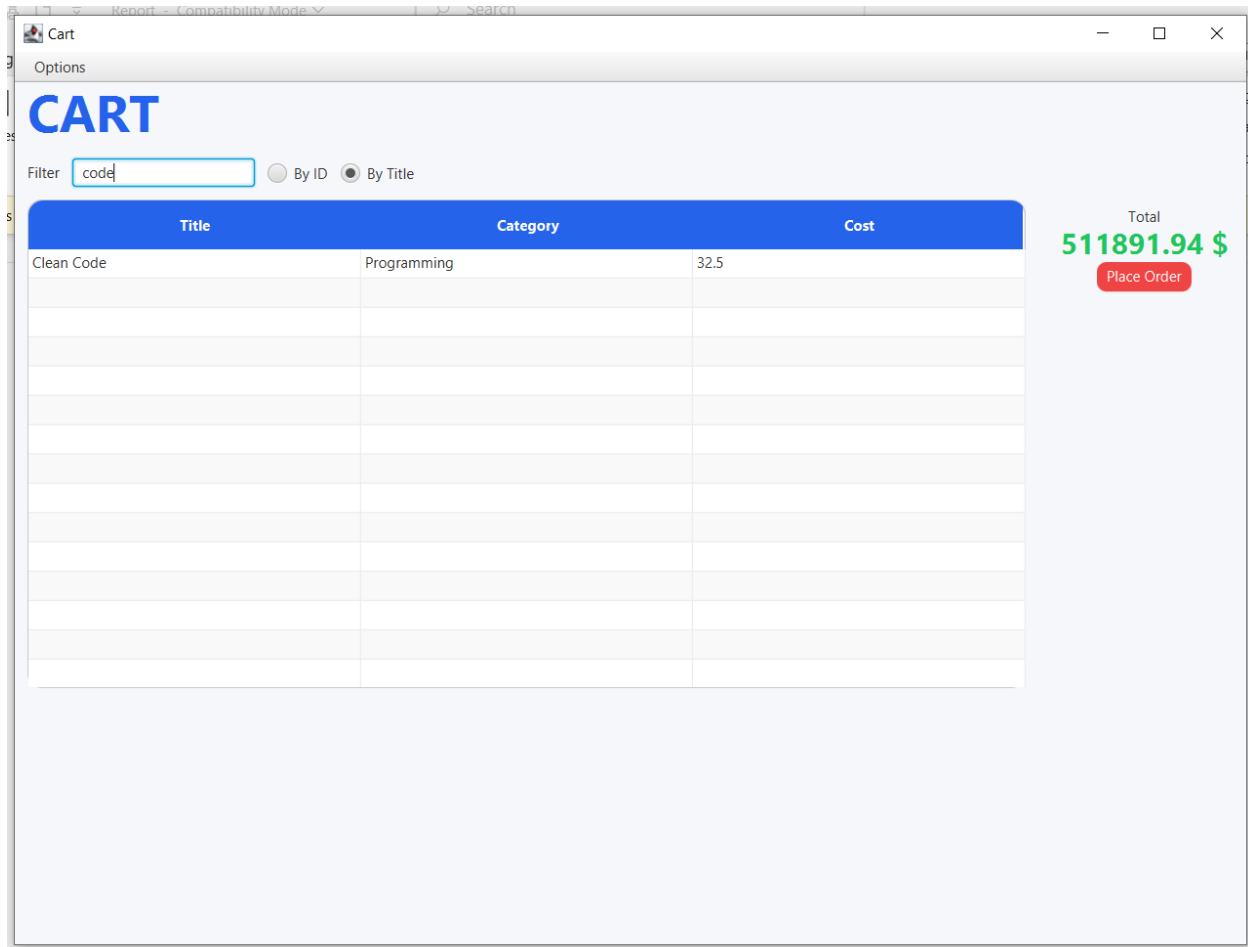


3.4.1 Filtering and Searching

Users can use the `tfFilter` text field to find specific items in the cart.



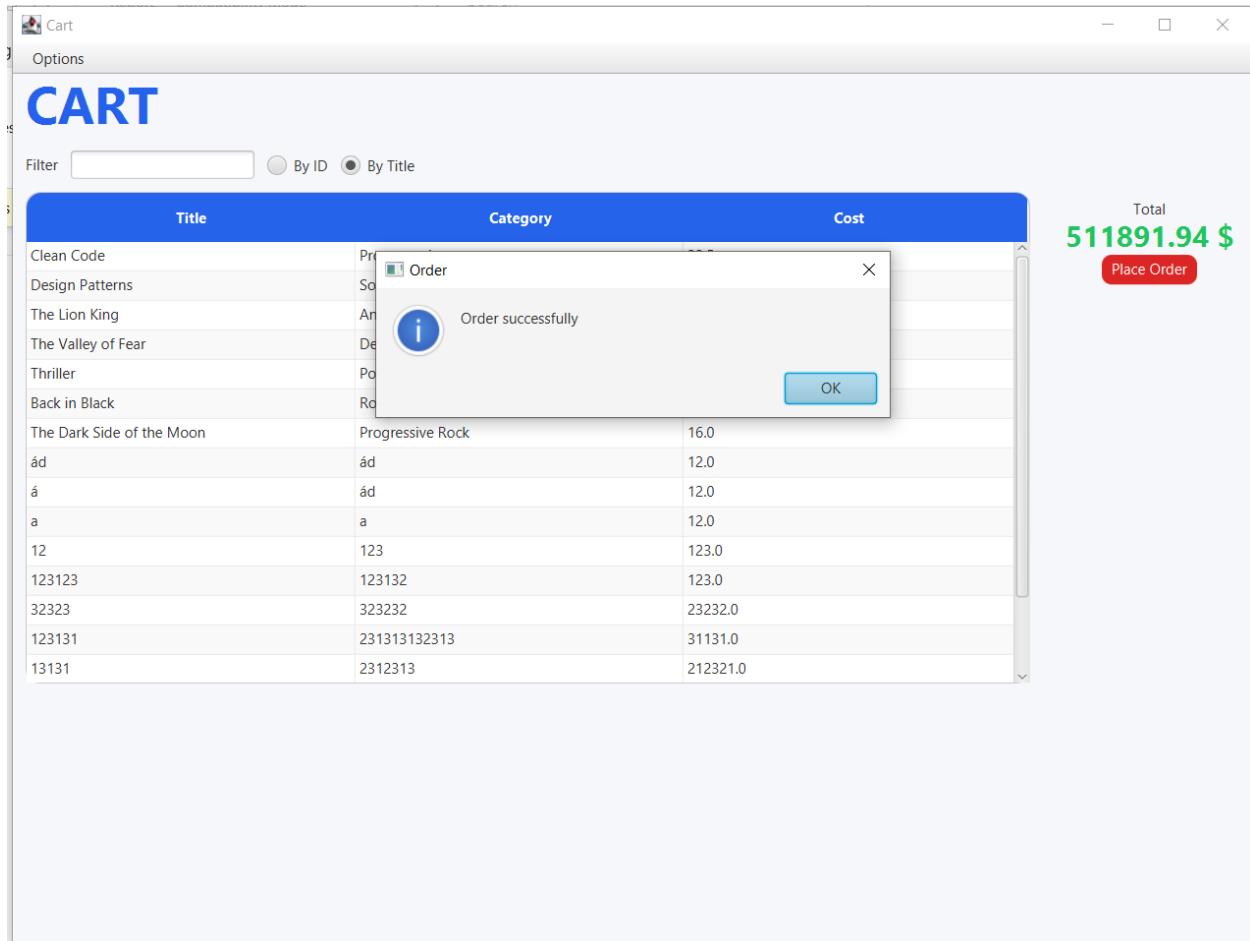
- **Filter by ID:** The system uses a Predicate to match the media's ID with the input.



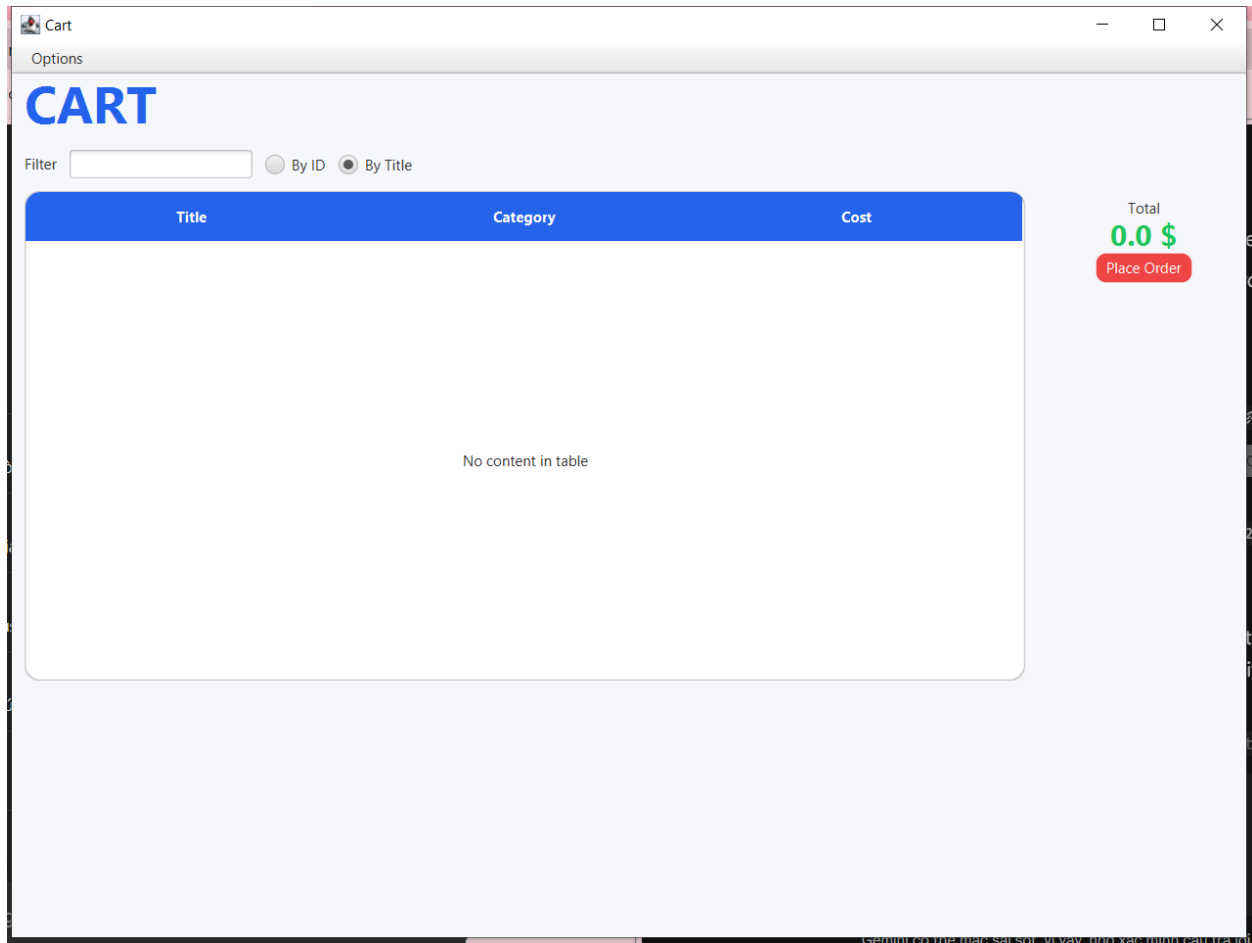
- **Filter by Title:** The system performs a case-insensitive search within the titles of items in the cart.

3.4.2 Order Placement

Clicking the **Place Order** button displays a confirmation alert .



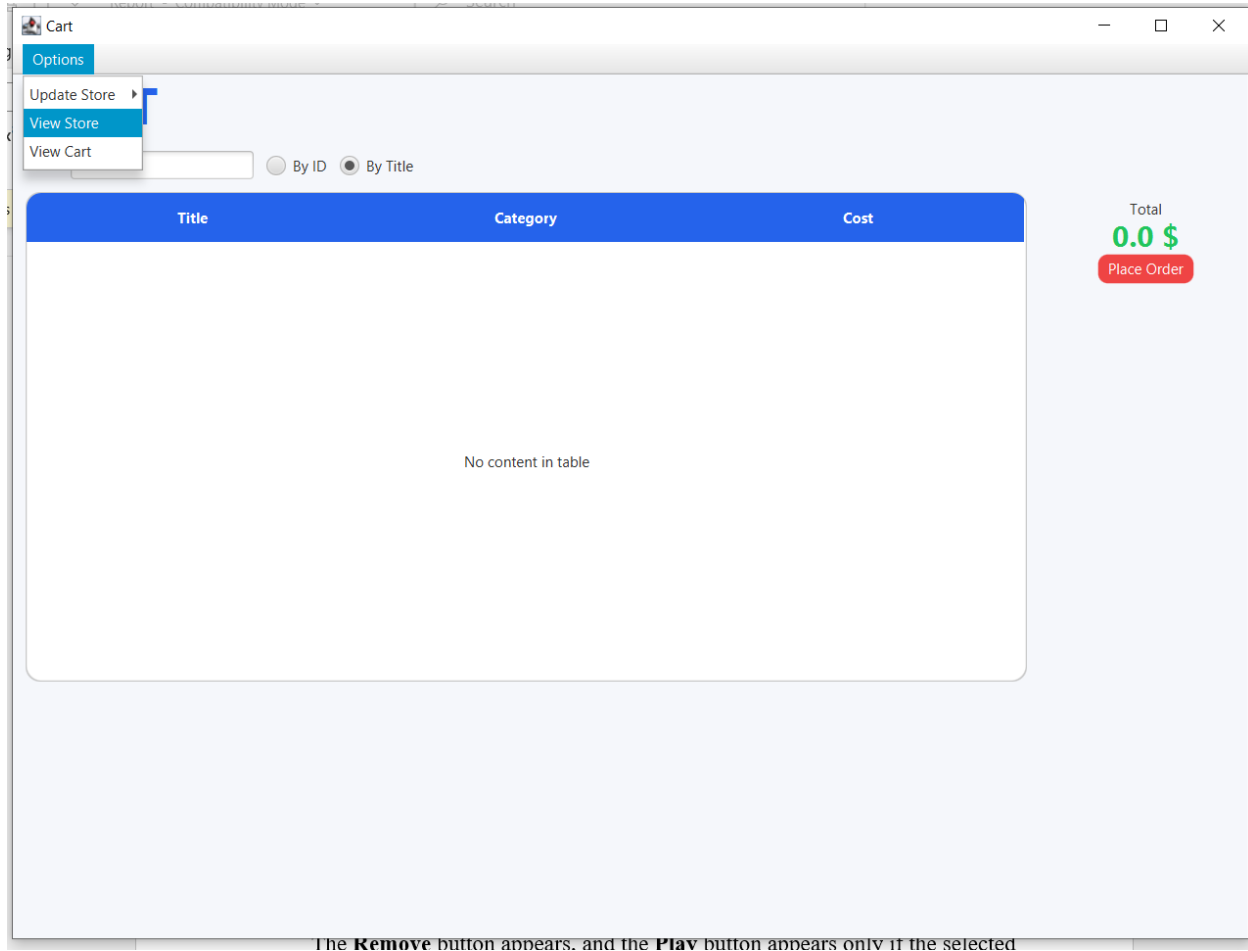
- Upon clicking "OK", the `cart.empty()` method is called to clear all items from the list.



- The total cost label is automatically updated to **0.00 \$**.

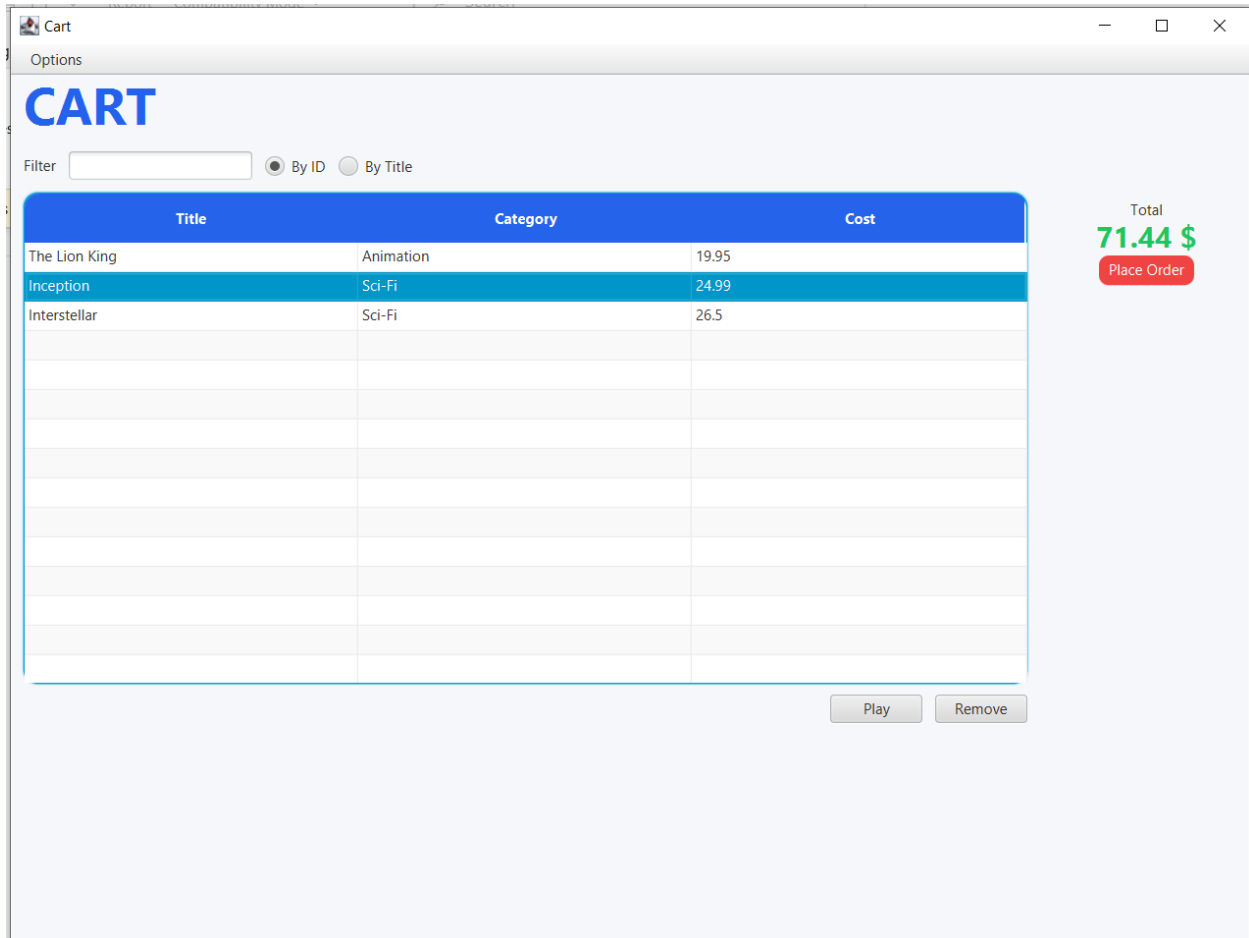
3.4.3 Returning to Store View

When the user clicks **View Store**, the **Store** interface is displayed again, allowing the user to continue adding items.

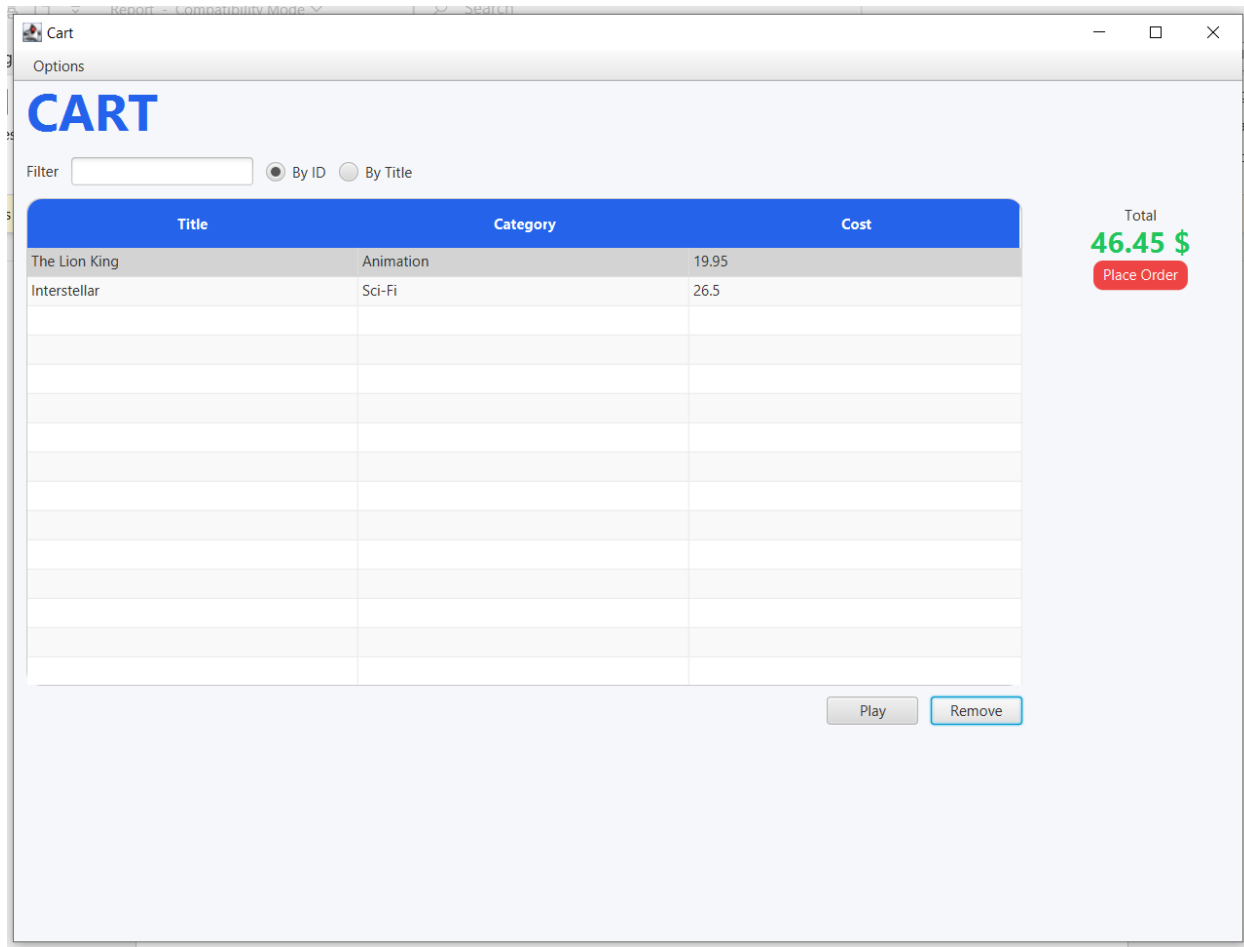


The **Remove** button appears, and the **Play** button appears only if the selected

3.4.4 Item Selection and Removal



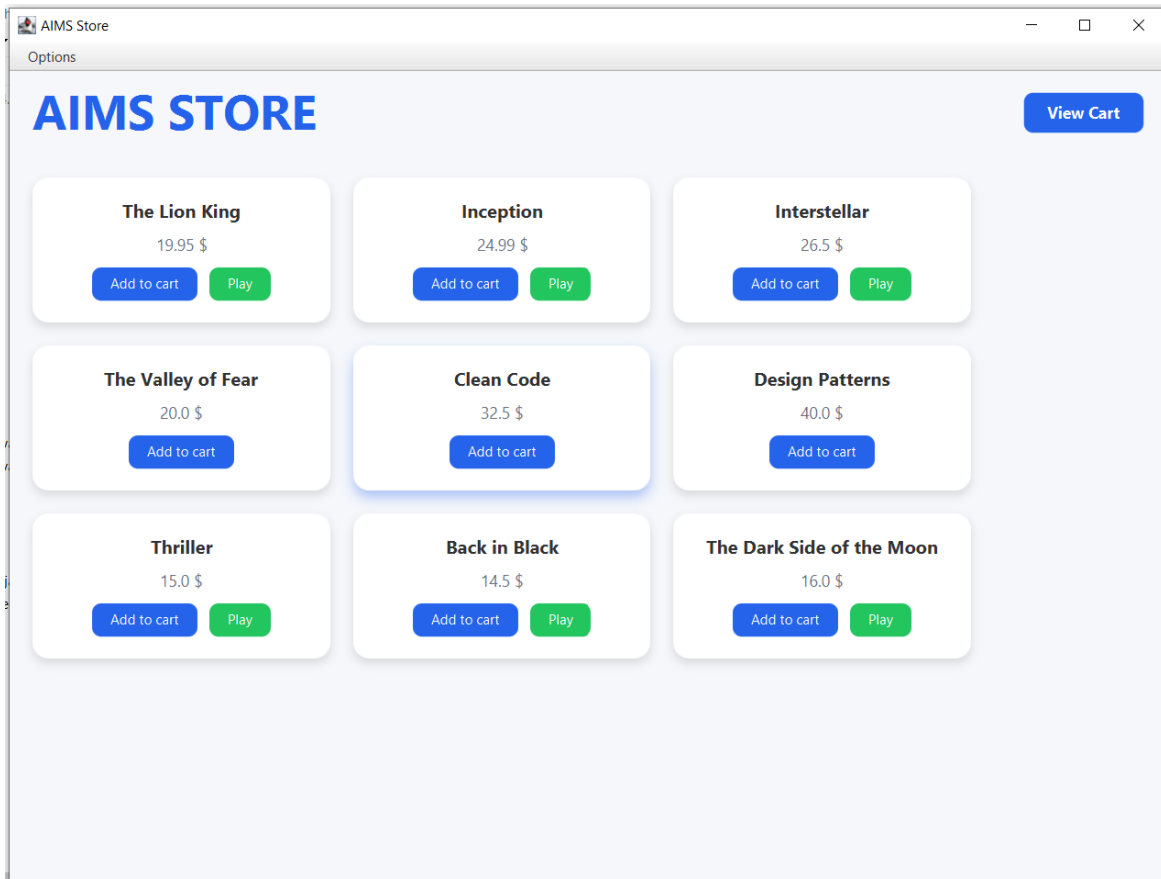
- **Selection:** When a user selects an item in the table, the button bar updates. The **Remove** button appears, and the **Play** button appears only if the selected item is playable .



- **Removal:** Clicking **Remove** calls `cart.removeMedia(selected)`, updating the UI and the total cost immediately .

3.5 Add to Store Functionality (Manager)

The **Options** menu allows the Store Manager to add new items to the inventory .



- **Add Book/CD/DVD:** Selecting these options opens specialized input screens like AddBookToStoreScreen.

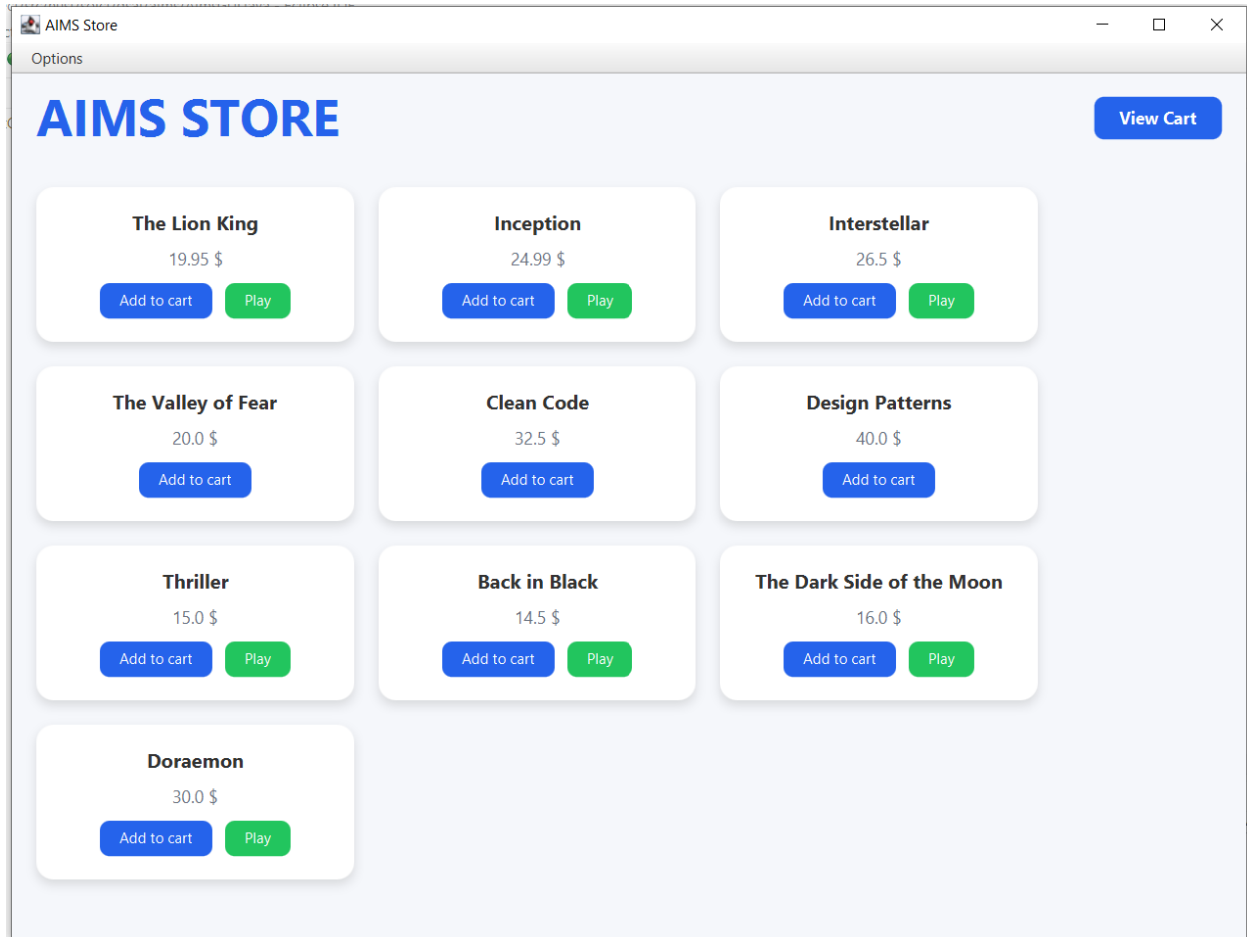
The screenshot shows a Windows application window titled "Add DVD". The window contains several text input fields for data entry:

- Title:** Doraemon
- Category:** Science Fiction
- Director:** (empty field)
- Length:** (empty field)
- Cost:** 30

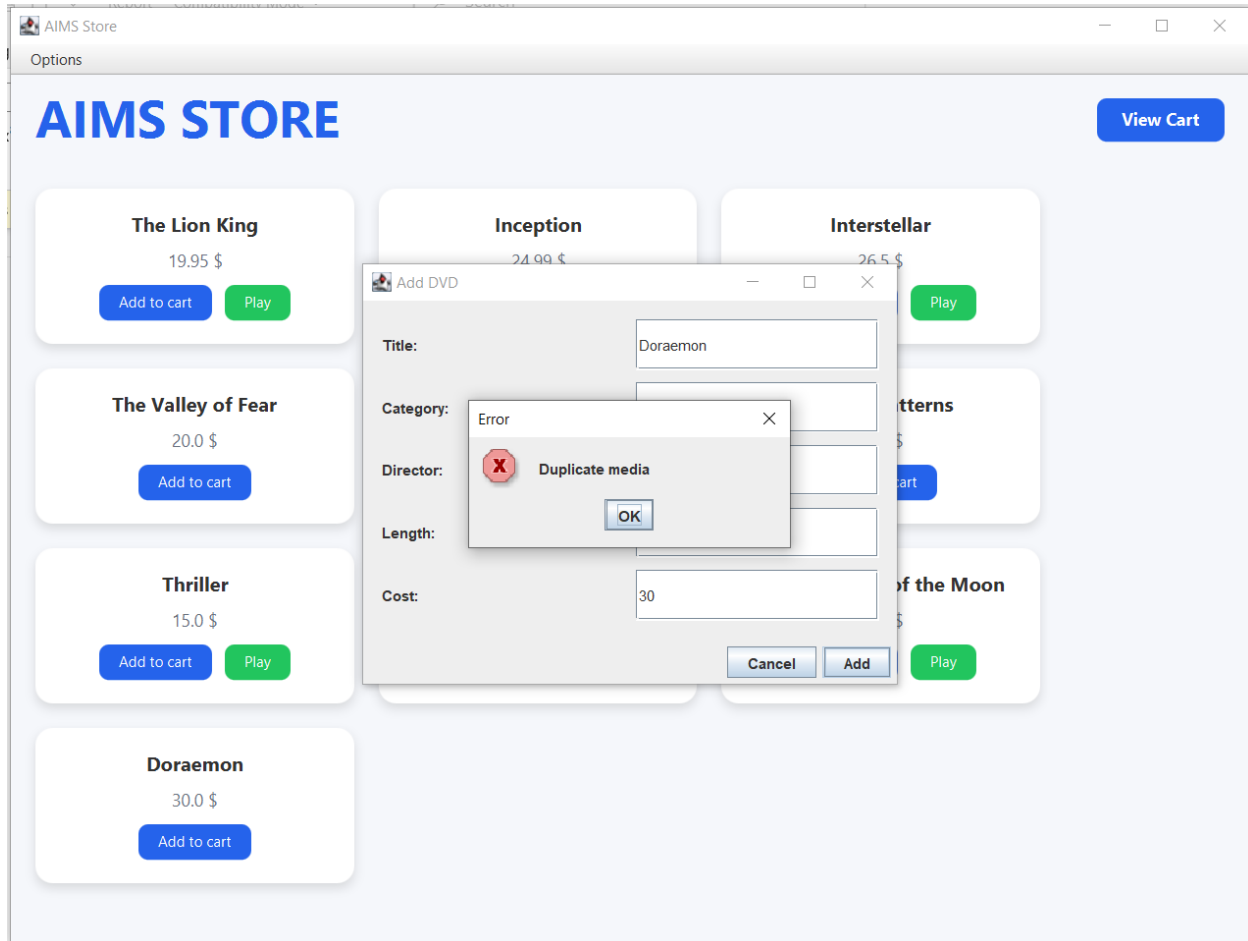
At the bottom right of the window are two buttons: "Cancel" and "Add".

A modal message box is overlaid on top of the "Add DVD" window. The message box is titled "Message" and contains an information icon (a lowercase 'i' inside a circle). The text inside the message box reads "Added DVD: Doraemon". Below the text is an "OK" button.

- **Validation:** The manager enters the Title, Category, Director, Length and Cost .



- **Success:** The new item is added to the store object, and the Store view is refreshed to show the new card .



- **Error:** If the item is a duplicate, an `IllegalArgumentException` is caught, and an error dialog is shown.

4. Exception Handling in the AIMS System

4.1 Purpose

Exception handling is a critical mechanism in the AIMS system designed to improve software robustness and user experience. The primary objectives are:

- **Preventing System Crashes:** By catching errors at runtime, the application avoids unexpected termination when encountering invalid data or logical failures.
- **Providing Meaningful Feedback:** Technical errors are intercepted and translated into user-friendly notifications via JavaFX Alert dialogs.
- **Ensuring Data Integrity:** The system validates state changes, such as preventing the addition of items beyond cart capacity or adding duplicate media.

4.2 Common Exception Scenarios

The system utilizes both custom and standard Java exceptions to manage various error states.

Scenario	Exception Type	Cause	Handling Action
Media Playback	PlayerException	Media length is non-positive.	Catch and display an "Error" alert dialog.
Full Cart	LimitExceededException	Attempting to add more than 20 items to the cart.	Notify the user that the cart limit has been reached.
Duplicate in Cart	IllegalArgumentException	Attempting to add an item already present in the cart.	Display a notification: "This item is already in your cart!"
Duplicate in Store	IllegalArgumentException	Manager attempts to add an existing item to the store .	Show an alert stating "Duplicate media".
Invalid Cost	IllegalArgumentException	Creating a media item with a negative cost value .	Throws exception to prevent object creation.

4.3 Implementing Exception Handling in Code

The logic for error handling is distributed between the backend classes (throwing exceptions) and the GUI controllers (catching and displaying them).

4.3.1 Handling Media Playback Errors

When a user attempts to play a DVD, CD, or Track, the system checks the `getLength()` method. If the length is invalid, a `PlayerException` is thrown.

```
try {  
    ((Playable) media).play();  
} catch (PlayerException ex) {  
    showError(ex.getMessage());  
}
```

4.3.2 Handling Cart Limit and Duplicate Items

The `addMedia` method in the `Cart` class ensures that neither the quantity limit is exceeded nor a duplicate item is added.

```
//Cart.java  
  
public void addMedia(Media media) throws LimitExceededException {  
    if (itemsOrdered.size() >= MAX_NUMBERS_ORDERED) {  
        throw new LimitExceededException("ERROR: The number of media  
has reached its limit");  
    }  
  
    if (itemsOrdered.contains(media)) {  
        throw new IllegalArgumentException("This item is already in  
your cart!");  
    }  
}
```

```
    }  
    itemsOrdered.add(media);  
}  
//MediaStore.java  
btnAdd.setOnAction(e -> {  
    try {  
        cart.addMedia(media);  
        showInfo("Added to cart");  
    } catch (LimitExceededException | IllegalArgumentException ex) {  
        showError(ex.getMessage());  
    }  
});
```

4.4 Benefits of Exception Handling

- **Stability:** The application remains operational even when users perform illegal actions.
- **Clarity:** Users receive precise feedback on why an action failed, such as a full cart or an unplayable file.
- **Maintainability:** By using specific exception types like `LimitExceededException`, developers can easily trace and debug logic errors.

4.5 Conclusion

The integration of a robust exception-handling framework is essential for the AIMS system. By proactively managing errors like playback failures and inventory limits, the system provides a professional and stable environment for both Customers and Store Managers, ensuring the application meets university-level software quality standards.